

14 – FreeRTOS

Real Time OS

Jérôme Landré – jerome.landre@ju.se

OUTLINE

- 1) Real-Time OS
- 2) FreeRTOS
- 3) Nucleo-64 STM32F401RE

1) REAL-TIME OS

- A **real-time Operating System** should be:
 - **Event-driven**
 - **Deterministic**
 - **Resource constrained**
 - Clock speed
 - Memory
 - Energy consumption

1) REAL-TIME OS

- A **real-time Operating System** can be:

- **Soft real-time response**

- Deterministic
- Predictable
- Non-critical
- Acceptable if deadline missed
- Examples: Video streaming, online gaming, sensor reading...

- **Hard real-time response**

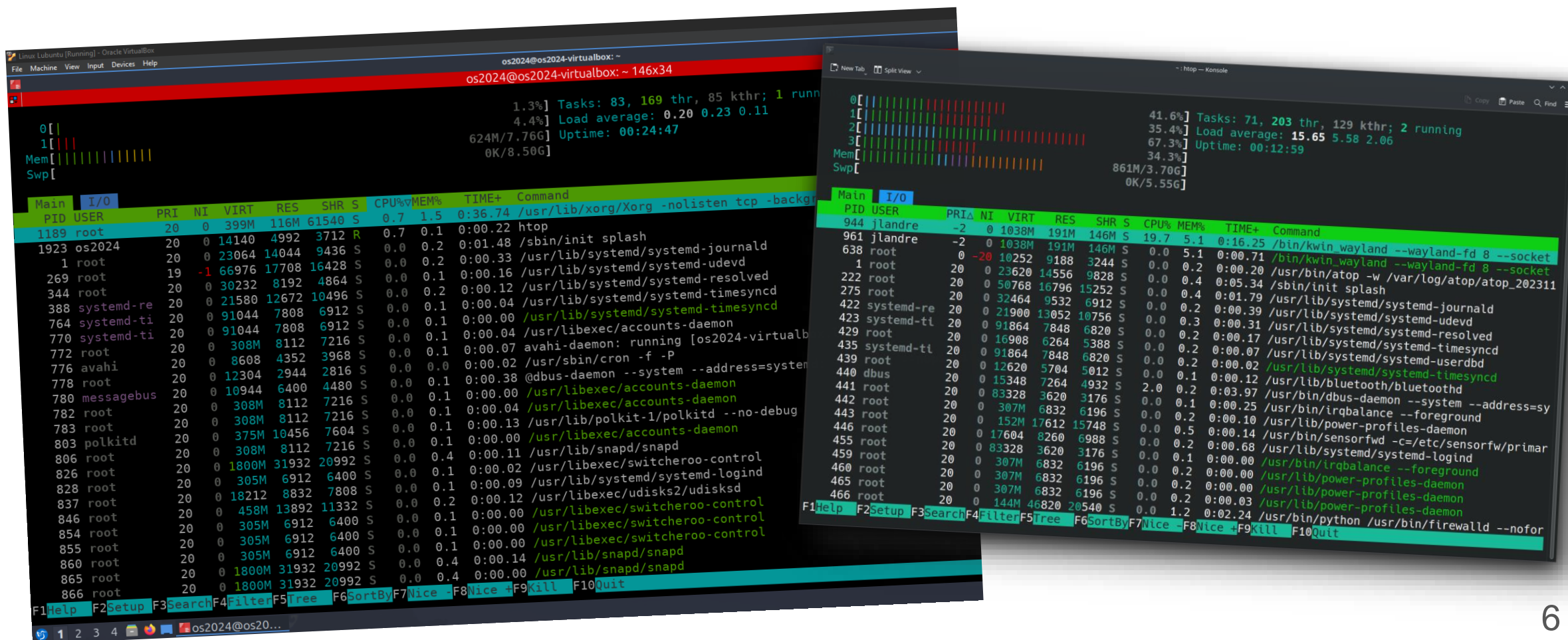
- Deterministic
- Predictable
- Critical
- Fatal system fault if deadline is not met
- Examples: aerospace, medical, car-driving applications...

1) REAL-TIME OS

- A **real-time Operating System** needs to guarantee deadlines of tasks to work properly:
 - Need to avoid “useless” software that takes a lot of CPU time
 - In general, RTOS don’t have a graphical interface: it takes too much CPU time
 - Round-Robin scheduler is not the best to guaranty that important tasks will be executed first
- => Use of **priority-based** (with Round Robin) **scheduling**
- => **Limit the use of memory** by **removing the graphical interface**

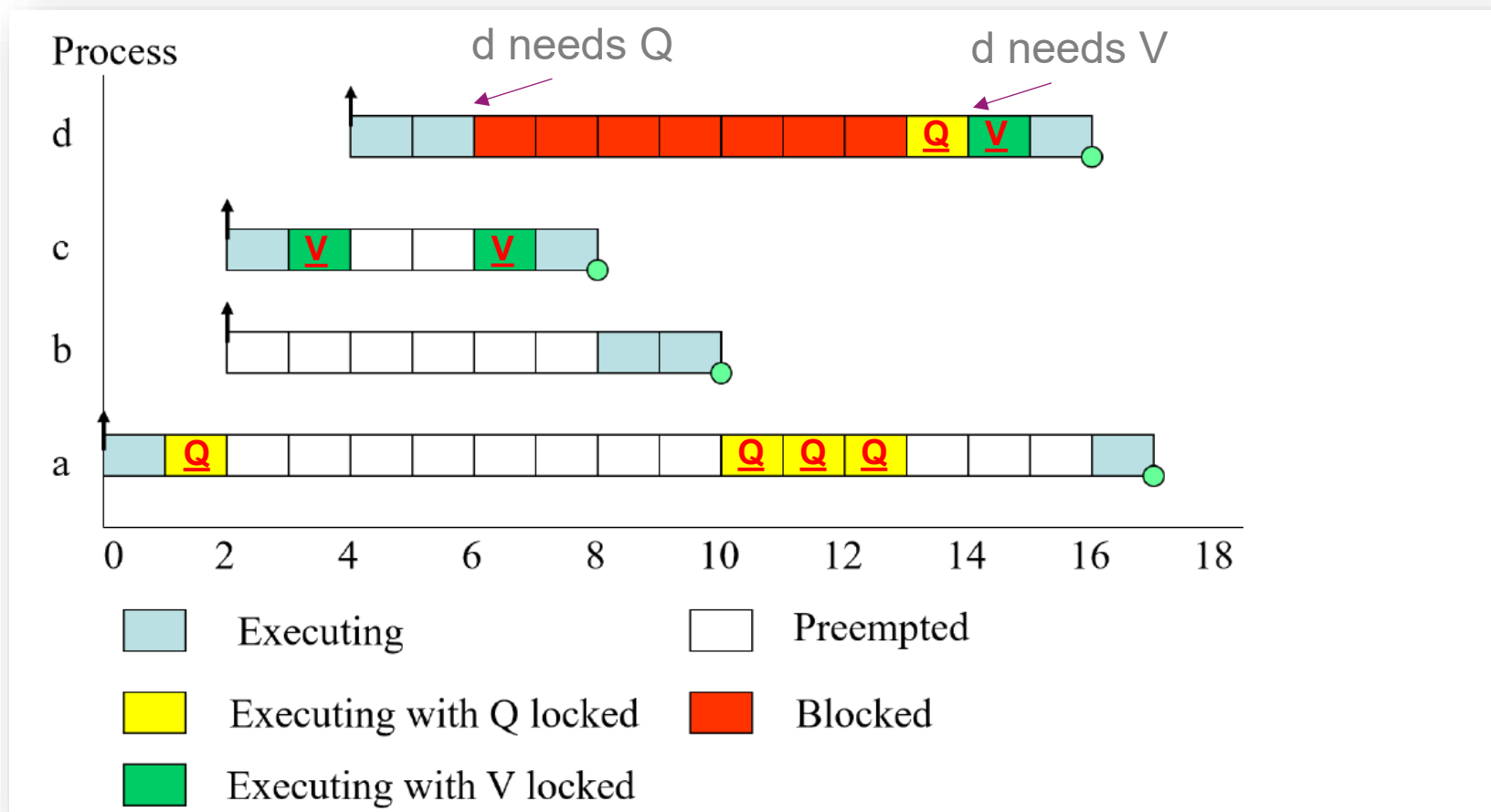
1) REAL-TIME OS

- Using htop: The **main CPU time** is for the **graphical interface!**



1) REAL-TIME OS

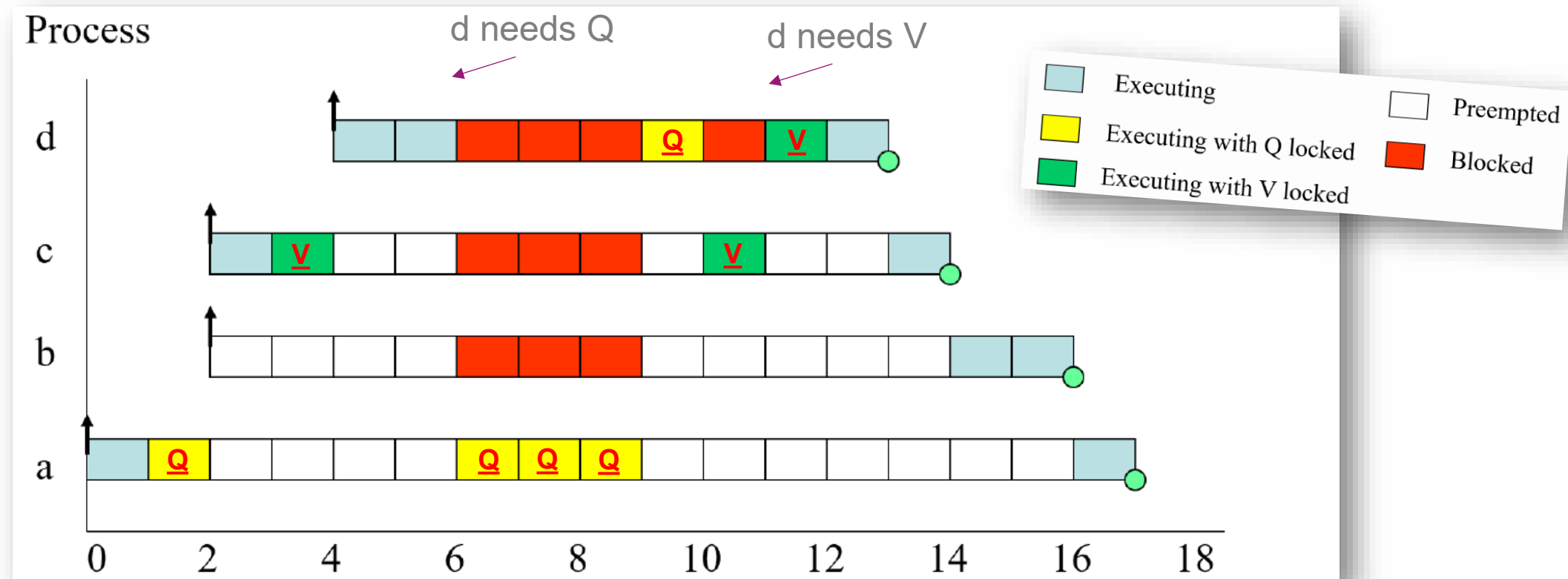
- When a task is blocked in a **real-time Operating System**, if no priority inversion:



Task with priority:
a locks Q
c locks V
d needs Q: blocked
...

1) REAL-TIME OS

- When a task is blocked in a **real-time Operating System**, if priority inversion:
- If process p blocks process q, then p runs with q priority!



2) FREERTOS

FreeRTOS™

Real-time operating system for microcontrollers

Developed in partnership with the world's leading chip companies over an 18-year period, and now downloaded every 170 seconds, FreeRTOS is a market-leading real-time operating system (RTOS) for microcontrollers and small microprocessors. Distributed freely under the MIT open source license, FreeRTOS includes a kernel and a growing set of IoT libraries suitable for use across all industry sectors. FreeRTOS is built with an emphasis on reliability and ease of use.



2) FREERTOS

- FreeRTOS documentation, PDF manual:

1.1 Multitasking in Small Embedded Systems

1.1.1 About the FreeRTOS Kernel

FreeRTOS is a collection of C libraries comprised of a real-time kernel and a set of modular libraries that implement complementary functionality.

Richard Barry originally developed FreeRTOS around 2003. Real-Time Engineers Ltd, Richard's company, continued FreeRTOS development in close partnership with the world's leading chip companies until Amazon Web Services (AWS) took stewardship of FreeRTOS 2016. Richard now continues his work on FreeRTOS as a senior principal engineer within the AWS IoT team. FreeRTOS is MIT licensed open source code, available for any purpose. You don't have to be an AWS customer to benefit from AWS's stewardship!

The FreeRTOS kernel is ideally suited to deeply embedded real-time applications that run on microcontrollers or small microprocessors. This type of application typically includes a mix of both hard and soft real-time requirements.

2) FREERTOS

Soft real-time requirements state a time deadline—but breaching the deadline would not render the system useless. For example, responding to keystrokes too slowly might make a system seem annoyingly unresponsive without actually making it unusable.

Hard real-time requirements state a time deadline—and breaching the deadline would result in absolute failure of the system. For example, a driver's airbag has the potential to do more harm than good if it responded to crash sensor inputs too slowly.

2) FREERTOS

The FreeRTOS kernel is a real-time kernel (or real-time scheduler) that enables applications built on FreeRTOS to meet their hard real-time requirements. It enables applications to be organized as a collection of independent threads of execution. For example, on a processor that has only one core, only a single thread of execution can execute at any one time. The kernel decides which thread to execute by examining the priority assigned to each thread by the application designer. In the simplest case, the application designer could assign higher priorities to threads that implement hard real-time requirements and lower priorities to threads that implement soft real-time requirements. Allocating priorities in that way would ensure hard real-time threads always execute ahead of soft real-time threads, but priority assignment decisions are not always that simplistic.

1.1.3 A Note About Terminology

In FreeRTOS, each thread of execution is called a 'task'. There is no consensus on terminology within the embedded community, but I prefer 'task' to 'thread,' as thread can have a more specific meaning in some fields of application.

2) FREERTOS

- Idle time

The automatically created Idle task executes when there are no application tasks that require processing. The Idle task can measure spare processing capacity, perform background checks, or place the processor into a low-power mode.

- Flexible interrupt handling

Interrupt handlers can be kept very short by deferring processing to either a task created by the application writer or the automatically created RTOS daemon task (also known as the timer task).

- Mixed processing requirements

Simple design patterns can achieve a mix of periodic, continuous, and event-driven processing within an application. In addition, hard and soft real-time requirements can be met by selecting appropriate task and interrupt priorities.

2) FREERTOS

1.1.5 FreeRTOS Kernel Features

The FreeRTOS kernel has the following standard features:

- Pre-emptive or co-operative operation
- Optional time-slicing
- Very flexible task priority assignment
- Flexible, fast and light-weight task notification mechanism
- Queues
- Binary semaphores
- Counting semaphores
- Mutexes
- Recursive mutexes
- Software timers
- Event groups
- Stream buffers
- Message buffers
- Co-routines (deprecated)
- Tick hook functions
- Idle hook functions
- Stack overflow checking
- Trace macros
- Task run-time statistics gathering
- Optional commercial licensing and support
- Full interrupt nesting model (for some architectures)
- A tick-less capability for extreme low power applications (for some architectures)
- Memory Protection Unit support for isolating tasks and increasing application safety (for some architectures)
- Software managed interrupt stack when appropriate (this can help save RAM)
- The ability to create RTOS objects using either statically or dynamically allocated memory

2) FREERTOS

1.1.6 Licensing, and The FreeRTOS, OpenRTOS, and SafeRTOS Family

The **FreeRTOS** MIT open source license is designed to ensure:

- FreeRTOS can be used in commercial applications.
- FreeRTOS itself remains freely available to everybody.
- FreeRTOS users retain ownership of their intellectual property.

See <https://www.FreeRTOS.org/license> for the latest open source license information.

OpenRTOS is a commercially licensed version of FreeRTOS provided under license from Amazon Web Services by a third party.

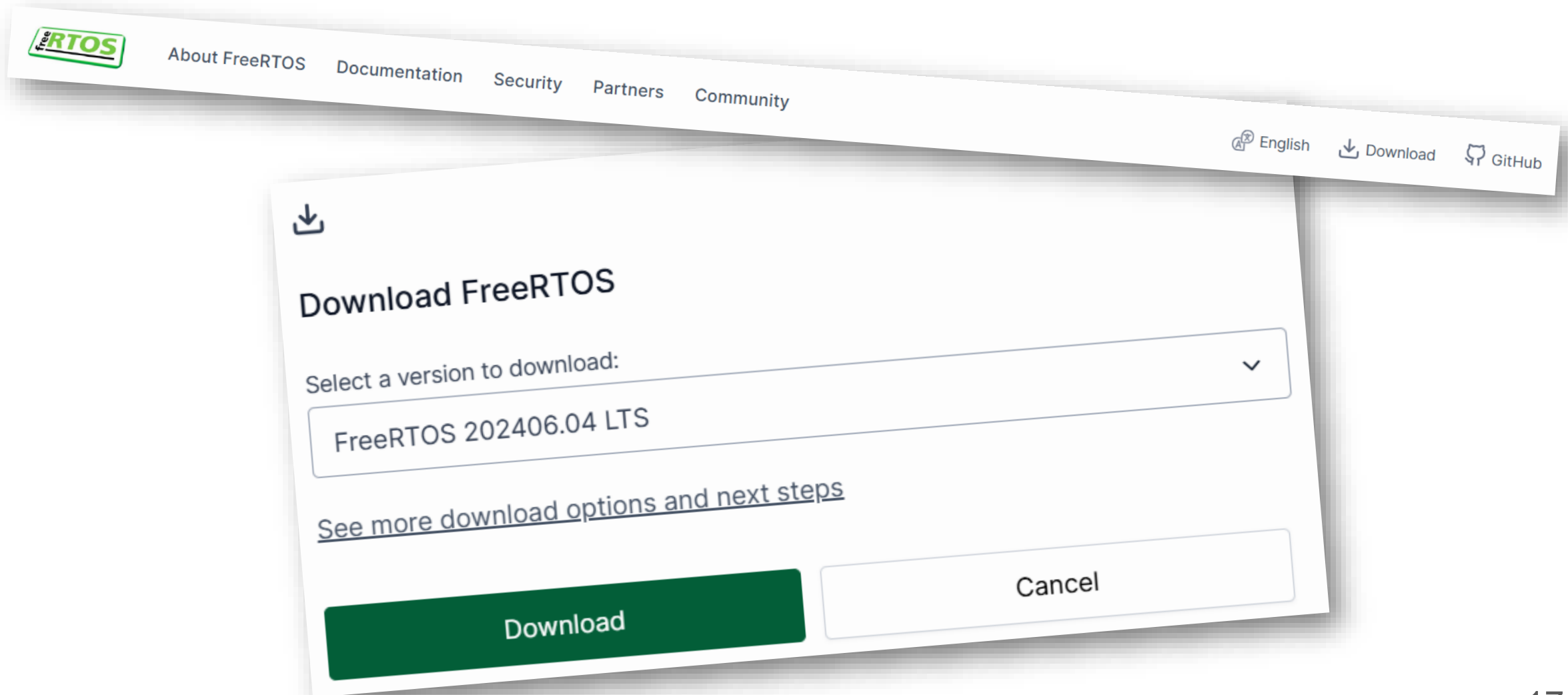
SafeRTOS shares the same usage model as FreeRTOS, but has been developed in accordance with the practices, procedures, and processes necessary to claim compliance with various internationally recognized safety related standards.

2) FREERTOS

- FreeRTOS is written mainly in C with several parts in assembly.
- The code is easy to read, and the functions syntax are “close to” POSIX threads
- A blinking example for two threads is given →

```
1  #include <FreeRTOS.h>
2  #include <task.h>
3  // Define your LED pin numbers here
4  #define RED_LED_PIN    12
5  #define GREEN_LED_PIN  13
6  void Task_RedLED(void *pvParameters) {
7      (void)pvParameters;
8      for (;;) {
9          // Blink the red LED every 400 ms
10         HAL_GPIO_WritePin(LD2_GPIO_Port, RED_LED_PIN, GPIO_PIN_RESET);
11         vTaskDelay(pdMS_TO_TICKS(200)); // Delay of 200ms
12         HAL_GPIO_WritePin(LD2_GPIO_Port, RED_LED_PIN, GPIO_PIN_SET);
13         vTaskDelay(pdMS_TO_TICKS(200)); // Delay of 200ms
14     }
15 }
16 void Task_GreenLED(void *pvParameters) {
17     (void)pvParameters;
18     for (;;) {
19         // Blink the green LED every 300 ms
20         HAL_GPIO_WritePin(LD2_GPIO_Port, RED_LED_PIN, GPIO_PIN_RESET);
21         vTaskDelay(pdMS_TO_TICKS(150)); // Delay of 150ms
22         HAL_GPIO_WritePin(LD2_GPIO_Port, RED_GREEN_LED_PIN, GPIO_PIN_SET);
23         vTaskDelay(pdMS_TO_TICKS(150)); // Delay of 150ms
24     }
25 }
26 void setup() {
27     // Initialize your microcontroller's GPIO pins for the LEDs here
28     // Create tasks for red and green LEDs
29     xTaskCreate(Task_RedLED, "RedLED_Task", configMINIMAL_STACK_SIZE, NULL, tskIDLE_PRIORITY + 1, NULL);
30     xTaskCreate(Task_GreenLED, "GreenLED_Task", configMINIMAL_STACK_SIZE, NULL, tskIDLE_PRIORITY + 1, NULL);
31     // Start the scheduler
32     vTaskStartScheduler();
33 }
34 int main() {
35     // Initialize hardware and start FreeRTOS scheduler
36     setup();
37     return 0;
38 }
```


2) FREERTOS

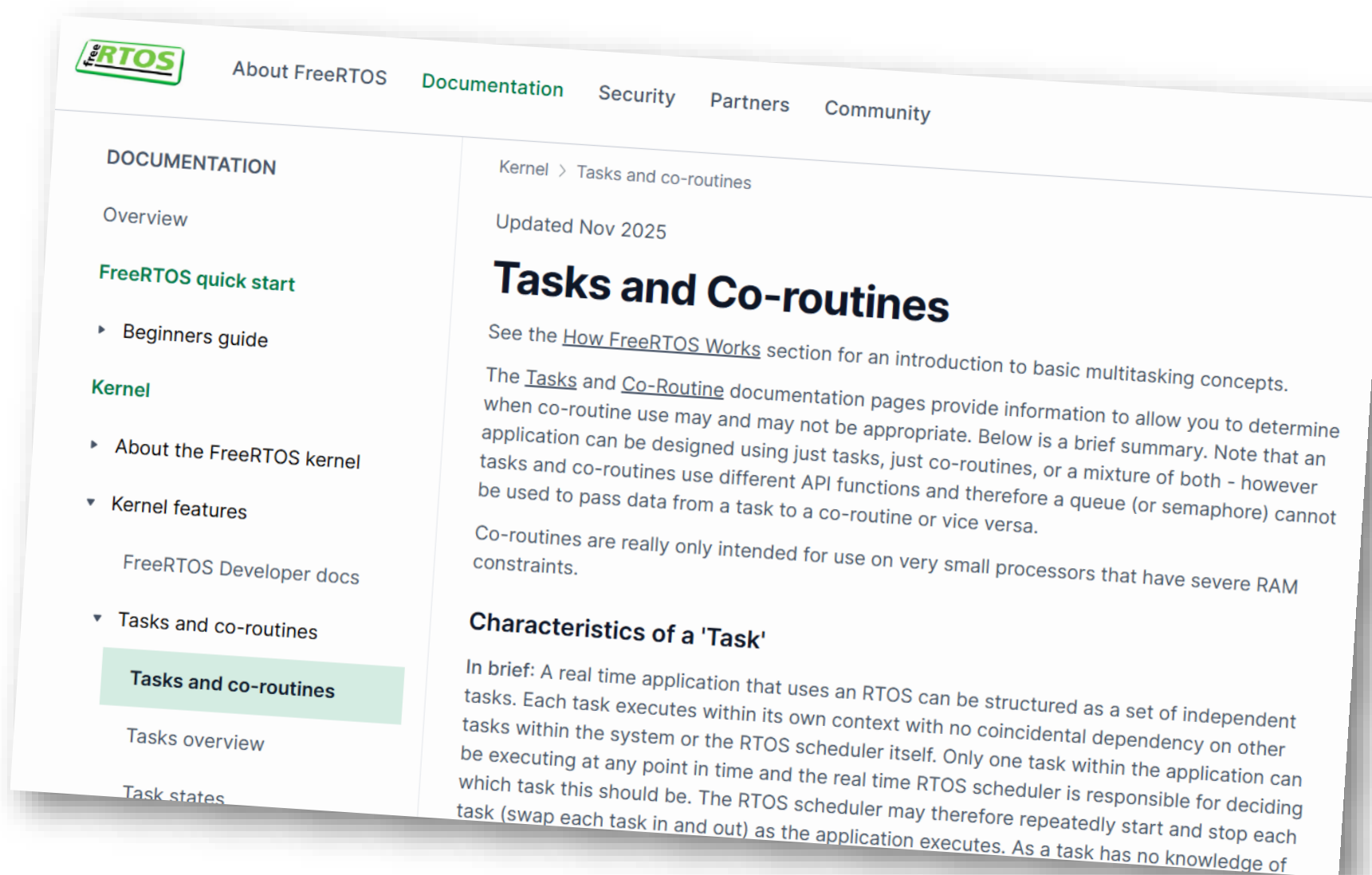


2) FREERTOS

- FreeRTOS supports many different architectures:

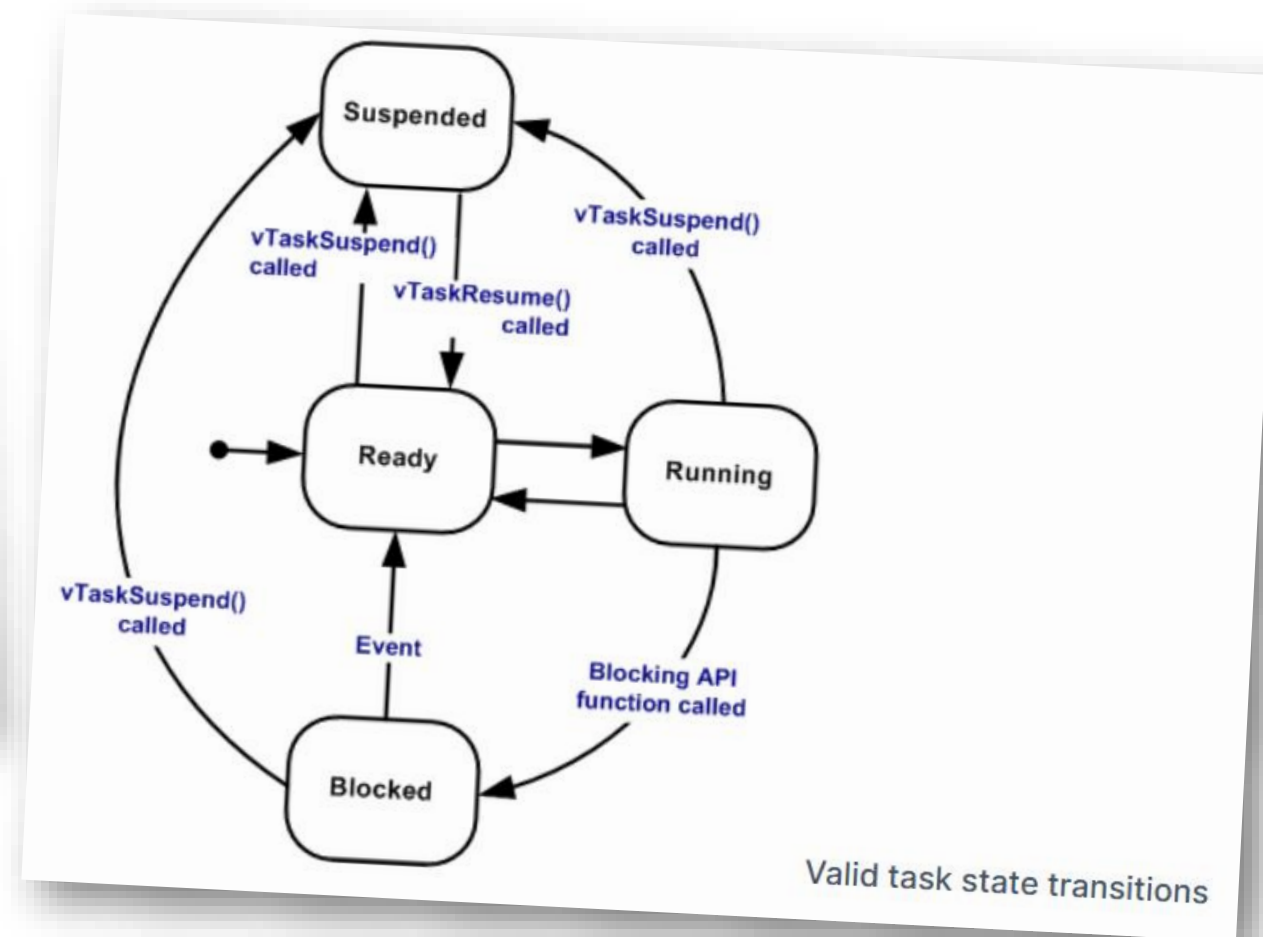
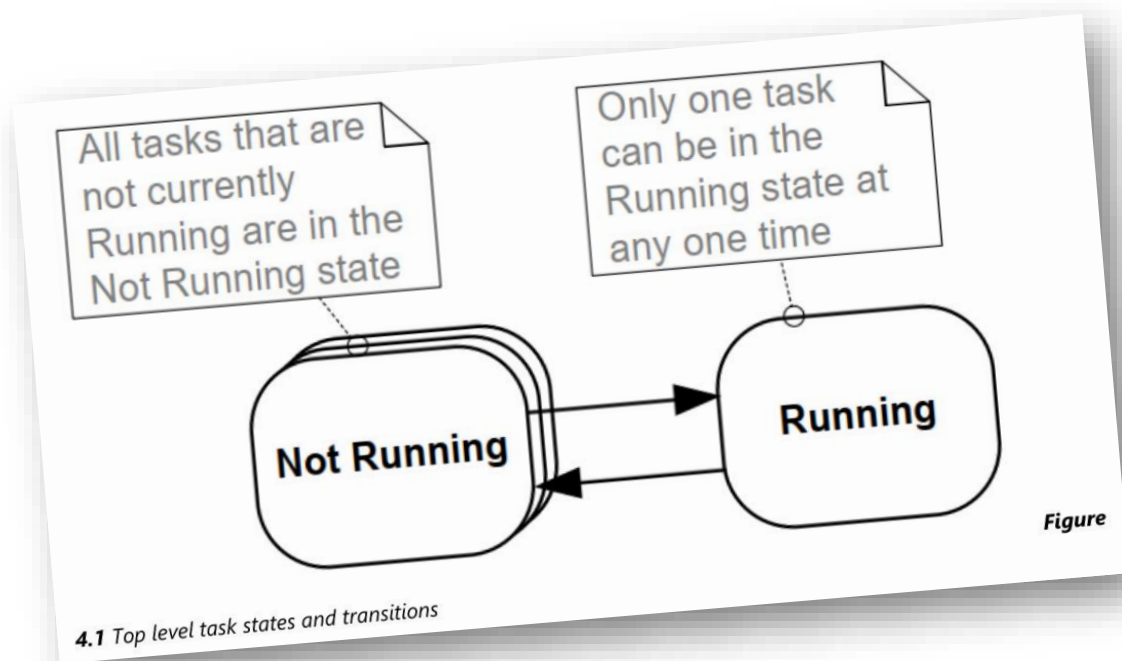
Hardware Partner	Supported Processor Families	Supported Tools
A		
<u>Altera</u>	Cyclone V SoC (ARM Cortex-A9), Nios II	Altera SoC EDS (ARM DS-5 with GCC), Nios II IDE with GCC
<u>ARMv8-M</u> Note this category is just for simulated targets. Other ARMv8-M targets are in their respective vendor categories.	ARM Cortex-M33 simulator	GCC (and ARMclang building the FreeRTOS ARMv8-M GCC port)
<u>Atmel</u>	SAMV7 (ARM Cortex-M7), SAM3 (ARM Cortex-M3), SAM4 (ARM Cortex-M4), SAMD20 (ARM Cortex-M0+), SAMA5 (ARM Cortex-A5), SAM7 (ARM7), SAM9 (ARM9), AT91, AVR and AVR32 UC3	IAR, GCC, Keil, Rowley CrossWorks
C		
<u>Cadence</u>	Tensilica Xtensa	XCC with the Xtensa Xplora IDE
<u>CEVA</u>	SensPro, SensPro2, CEVA-BX1, CEVA-BX2, CEVA-X1, CEVA-X2, CEVA-XC16, CEVA-XM6	LLVM

- FreeRTOS uses tasks and co-routines: <https://www.freertos.org/Documentation/02-Kernel/02-Kernel-features/01-Tasks-and-co-routines/00-Tasks-and-co-routines>



2) FREERTOS

- The different tasks states are defined here:



2) FREERTOS

- The tasks priorities can be changed in a FreeRTOS project:

“Each task is assigned a priority from 0 to (configMAX_PRIORITIES - 1), where configMAX_PRIORITIES is defined within FreeRTOSConfig.h.”

Task Priorities

Each task is assigned a priority from 0 to (configMAX_PRIORITIES - 1), where configMAX_PRIORITIES is defined within FreeRTOSConfig.h.

If the port in use implements a port optimised task selection mechanism that uses a 'count leading zeros' type instruction (for task selection in a single instruction) and configUSE_PORT_OPTIMISED_TASK_SELECTION is set to 1 in FreeRTOSConfig.h, then configMAX_PRIORITIES cannot be higher than 32. In all other cases configMAX_PRIORITIES can take any value within reason - but for reasons of RAM usage efficiency should be kept to the minimum value actually necessary.

Low priority numbers denote low priority tasks. The idle task has priority zero (tskIDLE_PRIORITY).

The FreeRTOS scheduler ensures that tasks in the Ready or Running state will always be given processor (CPU) time in preference to tasks of a lower priority that are also in the ready state. In other words, the task placed into the Running state is always the highest priority task that is able to run.

Any number of tasks can share the same priority. If configUSE_TIME_SLICING is not defined, or if configUSE_TIME_SLICING is set to 1, then Ready state tasks of equal priority will share the available processing time using a time sliced round robin scheduling scheme.

2) FREERTOS

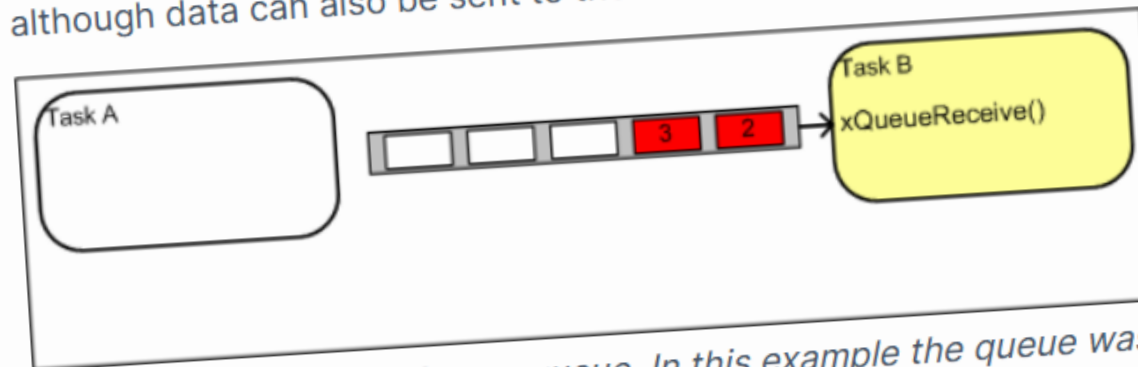
- FreeRTOS queues:

FreeRTOS queues

FreeRTOS Queues

[See also [Blocking on Multiple RTOS Objects](#)]

Queues are the primary form of intertask communications. They can be used to send messages between tasks, and between interrupts and tasks. In most cases they are used as thread safe FIFO (First In First Out) buffers with new data being sent to the back of the queue, although data can also be sent to the front.



Writing to and reading from a queue. In this example the queue was created to hold 5 items, and the queue never becomes full.

2) FREERTOS

- FreeRTOS supports semaphores (binary and counting):

Semaphore API functions permit a block time to be specified. The block time indicates the maximum number of 'ticks' that a task should enter the Blocked state when attempting to 'take' a semaphore, should the semaphore not be immediately available. If more than one task blocks on the same semaphore then the task with the highest priority will be the task that is unblocked the next time the semaphore becomes available.

Think of a binary semaphore as a queue that can only hold one item. The queue can therefore only be empty or full (hence binary). Tasks and interrupts using the queue don't care what the queue holds - they only want to know if the queue is empty or full. This mechanism can be exploited to synchronise (for example) a task with an interrupt.

2) FREERTOS

- FreeRTOS supports mutexes:

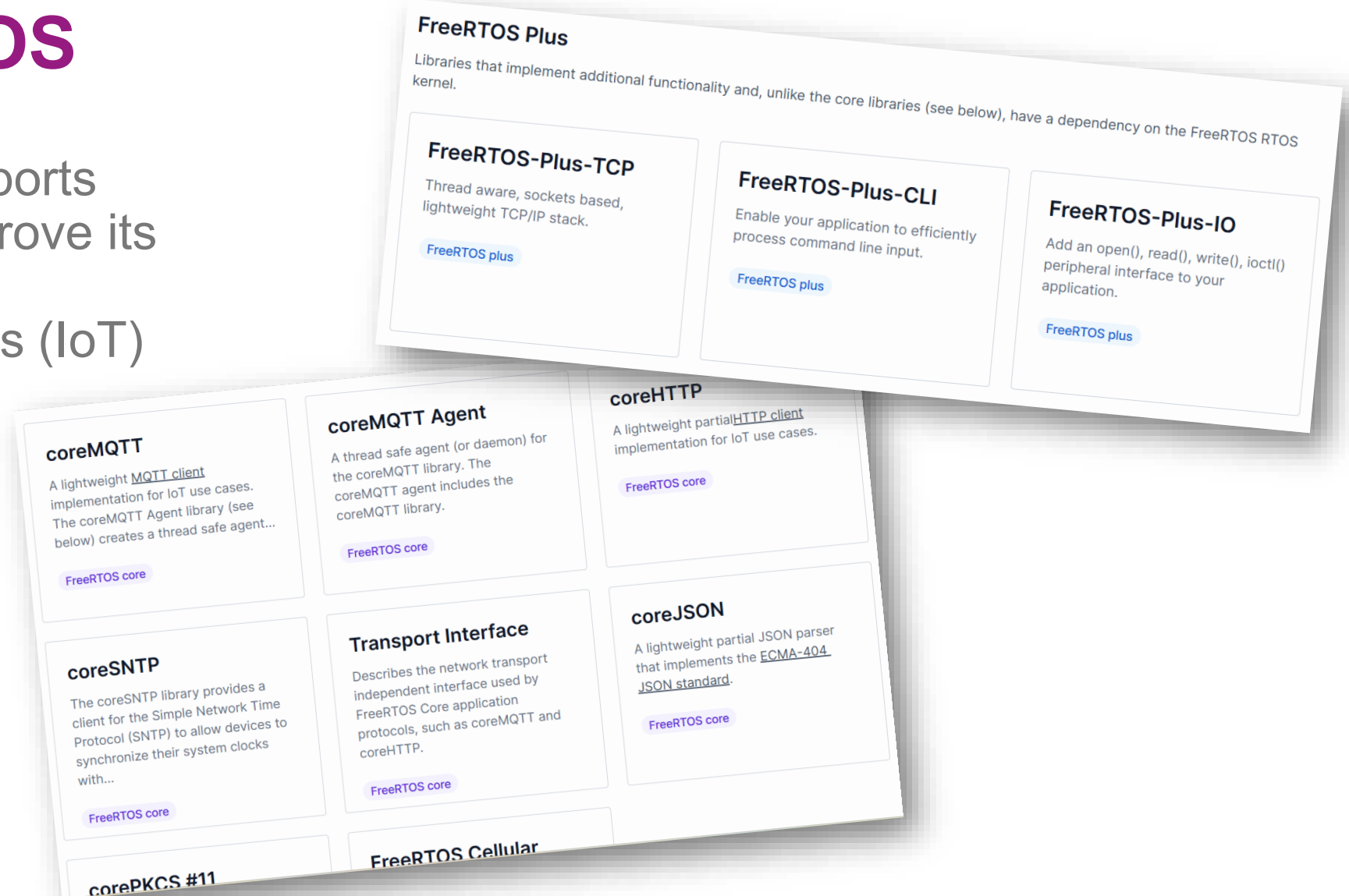
FreeRTOS Mutexes

Mutexes are binary semaphores that include a priority inheritance mechanism. Whereas binary semaphores are the better choice for implementing synchronisation (between tasks or between tasks and an interrupt), mutexes are the better choice for implementing simple mutual exclusion (hence 'MUT'ual 'EX'clusion).

When used for mutual exclusion the mutex acts like a token that is used to guard a resource. When a task wishes to access the resource it must first obtain ('take') the token. When it has finished with the resource it must 'give' the token back - allowing other tasks the opportunity to access the same resource.

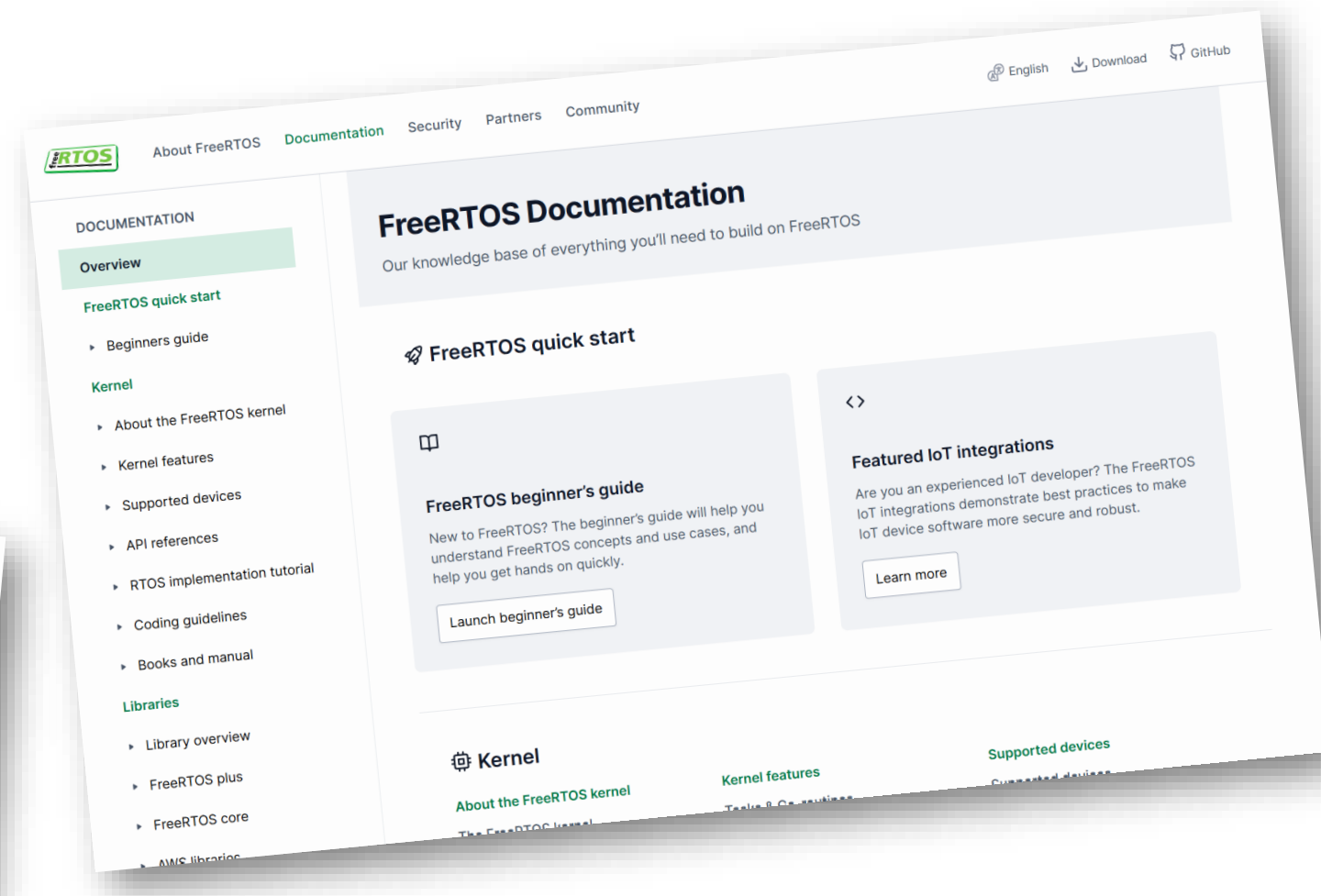
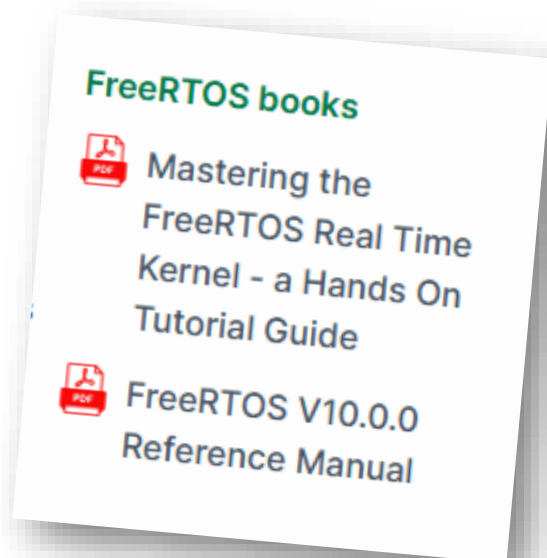
2) FREERTOS

- FreeRTOS supports **libraries** to improve its capacity for InternetOfThings (IoT) devices:



2) FREERTOS

- Building a project with FreeRTOS:



2) FREERTOS

- Building a project with FreeRTOS:

2.2.6 FreeRTOS Source Files Common to All Ports

tasks.c and list.c implement the core FreeRTOS kernel functionality and are always required. They are located directly in the FreeRTOS/Source directory, as shown in Figure 2.2. The same directory also contains the following optional source files:

- **queue.c**

queue.c provides both queue and semaphore services, as described later in this book. queue.c is nearly always required.

- **timers.c**

timers.c provides software timer functionality, as described later in this book. It only needs to be built if the application uses software timers.

FreeRTOS	
└─Source	
└─tasks.c	FreeRTOS source file - always required
└─list.c	FreeRTOS source file - always required
└─queue.c	FreeRTOS source file - nearly always required
└─timers.c	FreeRTOS source file - optional
└─event_groups.c	FreeRTOS source file - optional
└─stream_buffer.c	FreeRTOS source file - optional
└─croutine.c	FreeRTOS source file - optional and no longer maintained

- **event_groups.c**

event_groups.c provides event group functionality, as described later in this book. It only needs to be built if the application uses event groups.

- **stream_buffer.c**

stream_buffer.c provides both stream buffer and message buffer functionality, as described later in this book. It only needs to be built if the application uses stream or message buffers.

- **croutine.c**

croutine.c implements the FreeRTOS co-routine functionality. It only needs to be built if the application uses co-routines. Co-routines are intended for use on very small microcontrollers, are rarely used now. They are, therefore, no longer maintained, and their use is not recommended for new designs. Co-routines are not described in this book.

2) FREERTOS

```
int main( void )
{
    /* Perform any hardware setup necessary. */
    prvSetupHardware();

    /* --- APPLICATION TASKS CAN BE CREATED HERE --- */

    /* Start the created tasks running. */
    vTaskStartScheduler();

    /* Execution will only reach here if there was insufficient heap to
       start the scheduler. */
    for( ;; );
    return 0;
}
```

Listing 2.1 The template for a new main() function

2) FREERTOS

2.5.1 Data Types

Each port of FreeRTOS has a unique portmacro.h header file that contains (amongst other things) definitions for two port-specific data types: `TickType_t` and `BaseType_t`. The following list describes the macro or typedef used and the actual type:

- `TickType_t`

FreeRTOS configures a periodic interrupt called the tick interrupt.

The number of tick interrupts that have occurred since the FreeRTOS application started is called the *tick count*. The tick count is used as a measure of time.

The time between two tick interrupts is called the *tick period*. Times are specified as multiples of tick periods.

`TickType_t` is the data type used to hold the tick count value, and to specify times.

`TickType_t` can be an unsigned 16-bit type, an unsigned 32-bit type, or an unsigned 64-bit type, depending on the setting of `configTICK_TYPE_WIDTH_IN_BITS` in FreeRTOSConfig.h. The setting of `configTICK_TYPE_WIDTH_IN_BITS` is architecture dependent. FreeRTOS ports will also check if this setting is valid.

2) FREERTOS

- `BaseType_t`

This is always defined as the most efficient data type for the architecture. Typically, this is a 64-bit type on a 64-bit architecture, a 32-bit type on a 32-bit architecture, a 16-bit type on a 16-bit architecture, and an 8-bit type on an 8-bit architecture.

`BaseType_t` is generally used for return types that take only a very limited range of values, and for `pdTRUE/pdFALSE` type Booleans.

2) FREERTOS

4.2 Task Functions

Tasks are implemented as C functions. Tasks must implement the expected function prototype shown in Listing 4.1. which takes a void pointer parameter and returns void.

```
void vATaskFunction( void * pvParameters );
```

Listing 4.1 *The task function prototype*

Each task is a small program in its own right. It has an entry point, will normally run forever in an infinite loop, and does not exit. Listing 4.2 shows the structure of a typical task.

A FreeRTOS task must not be allowed to return from the function that implements it in any way. It must not contain a 'return' statement and must not be allowed to execute past the end of its implementing function. If a task is no longer required, it should be explicitly deleted as demonstrated in Listing 4.2.


```
void vTaskFunction( void * pvParameters )
{
    /*
     * Stack-allocated variables can be declared normally when inside a function.
     * Each instance of a task created using this example function will have its
     * own separate instance of lStackVariable allocated on the task's stack.
     */
    long lStackVariable = 0;

    /*
     * In contrast to stack allocated variables, variables declared with the
     * `static` keyword are allocated to a specific location in memory by the linker.
     * This means that all tasks calling vTaskFunction will share the same
     * instance of lStaticVariable.
     */
    static long lStaticVariable = 0;

    for( ;; )
    {
        /* The code to implement the task functionality will go here. */
    }

    /*
     * If the task implementation ever exits the above loop, then the task
     * must be deleted before reaching the end of its implementing function.
     * When NULL is passed as a parameter to the vTaskDelete() API function,
     * this indicates that the task to be deleted is the calling (this) task.
     */
    vTaskDelete( NULL );
}
```

Listing 4.2 The structure of a typical task function

2) FREERTOS

```
BaseType_t xTaskCreate( TaskFunction_t pvTaskCode,  
                        const char * const pcName,  
                        configSTACK_DEPTH_TYPE usStackDepth,  
                        void * pvParameters,  
                        UBaseType_t uxPriority,  
                        TaskHandle_t * pxCreatedTask );
```

Listing 4.3 The `xTaskCreate()` API function prototype

- **pvTaskCode**

Tasks are simply C functions that never exit and, as such, are normally implemented as an infinite loop. The **pvTaskCode** parameter is simply a pointer to the function that implements the task (in effect, just the function's name).

- **pcName**

A descriptive name for the task. FreeRTOS does not use this in any way and it is included purely as a debugging aid. Identifying a task by a human-readable name is much simpler than identifying it by its handle.

2) FREERTOS

```
BaseType_t xTaskCreate( TaskFunction_t pvTaskCode,  
                        const char * const pcName,  
                        configSTACK_DEPTH_TYPE usStackDepth,  
                        void * pvParameters,  
                        UBaseType_t uxPriority,  
                        TaskHandle_t * pxCreatedTask );
```

Listing 4.3 The `xTaskCreate()` API function prototype

- `usStackDepth`

Specifies the size of the stack to allocate for use by the task. Use `xTaskCreateStatic()` instead of `xTaskCreate()` to use pre-allocated memory instead of dynamically allocated memory.

Note the value specifies the number of words the stack can hold, not the number of bytes. For example, if the stack is 32-bits wide and `usStackDepth` is 128, then `xTaskCreate()` allocates 512 bytes of stack space (128 * 4 bytes).

2) FREERTOS

```
BaseType_t xTaskCreate( TaskFunction_t pvTaskCode,  
                        const char * const pcName,  
                        configSTACK_DEPTH_TYPE usStackDepth,  
                        void * pvParameters,  
                        UBaseType_t uxPriority,  
                        TaskHandle_t * pxCreatedTask );
```

Listing 4.3 The `xTaskCreate()` API function prototype

- `pvParameters`

Functions that implement tasks accept a single void pointer (`void *`) parameter. `pvParameters` is the value passed into the task using that parameter.

- `uxPriority`

Defines the task's priority. 0 is the lowest priority and `(configMAX_PRIORITIES - 1)` is the highest priority. [Section 4.5](#) describes the user defined `configMAX_PRIORITIES` constant.

If a `uxPriority` greater than `(configMAX_PRIORITIES - 1)` is defined, it will be capped to `(configMAX_PRIORITIES - 1)`.

2) FREERTOS

```
BaseType_t xTaskCreate( TaskFunction_t pvTaskCode,  
                        const char * const pcName,  
                        configSTACK_DEPTH_TYPE usStackDepth,  
                        void * pvParameters,  
                        UBaseType_t uxPriority,  
                        TaskHandle_t * pxCreatedTask );
```

Listing 4.3 The `xTaskCreate()` API function prototype

- `pxCreatedTask`

Pointer to a location to store a handle to the created task. This handle may be used in future API calls to, for example, change the task's priority or delete the task.

`pxCreatedTask` is an optional parameter and may be set to NULL if the task's handle is not required.

2) FREERTOS

```
BaseType_t xTaskCreate( TaskFunction_t pvTaskCode,  
                        const char * const pcName,  
                        configSTACK_DEPTH_TYPE usStackDepth,  
                        void * pvParameters,  
                        UBaseType_t uxPriority,  
                        TaskHandle_t * pxCreatedTask );
```

Listing 4.3 The `xTaskCreate()` API function prototype

- Return values

There are two possible return values:

- `pdPASS`

This indicates the task was created successfully.

- `pdFAIL`

```
void vTask1( void * pvParameters )
{
    /* ulCount is declared volatile to ensure it is not optimized out. */
    volatile unsigned long ulCount;

    for( ;; )
    {
        /* Print out the name of the current task task. */
        vPrintLine( "Task 1 is running" );

        /* Delay for a period. */
        for( ulCount = 0; ulCount < mainDELAY_LOOP_COUNT; ulCount++ )
        {
            /*
             * This loop is just a very crude delay implementation. There is
             * nothing to do in here. Later examples will replace this crude
             * loop with a proper delay/sleep function.
             */
        }
    }
}
```

```
void vTask2( void * pvParameters )
{
    /* ulCount is declared volatile to ensure it is not optimized out. */
    volatile unsigned long ulCount;

    /* As per most tasks, this task is implemented in an infinite loop. */
    for( ;; )
    {
        /* Print out the name of this task. */
        vPrintLine( "Task 2 is running" );

        /* Delay for a period. */
        for( ulCount = 0; ulCount < mainDELAY_LOOP_COUNT; ulCount++ )
        {
            /*
             * This loop is just a very crude delay implementation. There is
             * nothing to do in here. Later examples will replace this crude
             * loop with a proper delay/sleep function.
             */
        }
    }
}
```

```
int main( void )
{
    /*
     * Variables declared here may no longer exist after starting the FreeRTOS
     * scheduler. Do not attempt to access variables declared on the stack used
     * by main() from tasks.
     */

    /*
     * Create one of the two tasks. Note that a real application should check
     * the return value of the xTaskCreate() call to ensure the task was
     * created successfully.
     */
    xTaskCreate( vTask1, /* Pointer to the function that implements the task.*/
                "Task 1", /* Text name for the task. */
                1000, /* Stack depth in words. */
                NULL, /* This example does not use the task parameter. */
                1, /* This task will run at priority 1. */
                NULL ); /* This example does not use the task handle. */

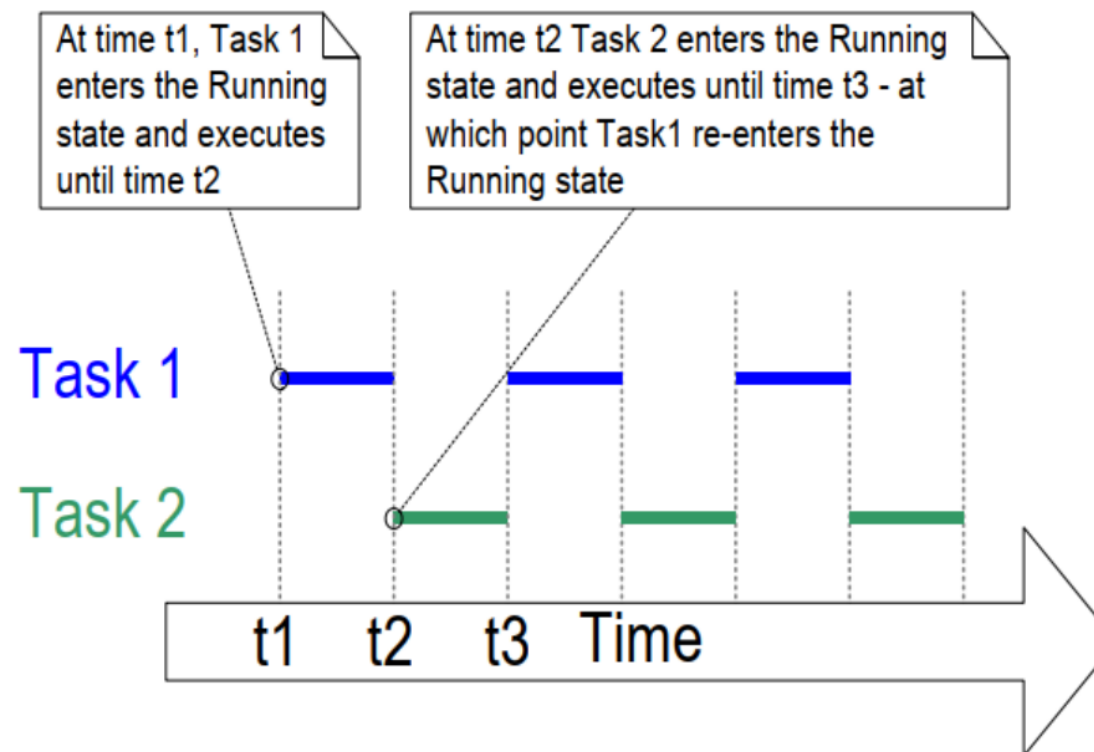
    /* Create the other task in exactly the same way and at the same priority.*/
    xTaskCreate( vTask2, "Task 2", 1000, NULL, 1, NULL );

    /* Start the scheduler so the tasks start executing. */
    vTaskStartScheduler();

    /*
     * If all is well main() will not reach here because the scheduler will now
     * be running the created tasks. If main() does reach here then there was
     * not enough heap memory to create either the idle or timer tasks
     * (described later in this book). Chapter 3 provides more information on
     * heap memory management.
     */
    for( ;; );
}
```

2) FREERTOS

```
C:\Temp>rtosdemo
Task 1 is running
Task 2 is running
Task 1 is running
Task 2 is running
Task 1 is running
Task 2 is running
Task 1 is running
Task 2 is running
Task 1 is running
Task 2 is running
Task 1 is running
Task 2 is running
Task 1 is running
Task 2 is running
Task 1 is running
Task 2 is running
```



Figure

4.3 The actual execution pattern of the two Example 4.1 tasks

2) FREERTOS

```
C:\Temp>rtosdemo
Task 1 is running
Task 2 is running
Task 1 is running
Task 2 is running
Task 1 is running
Task 2 is running
Task 1 is running
Task 2 is running
Task 1 is running
Task 2 is running
Task 1 is running
Task 2 is running
Task 1 is running
Task 2 is running
Task 1 is running
Task 2 is running
```

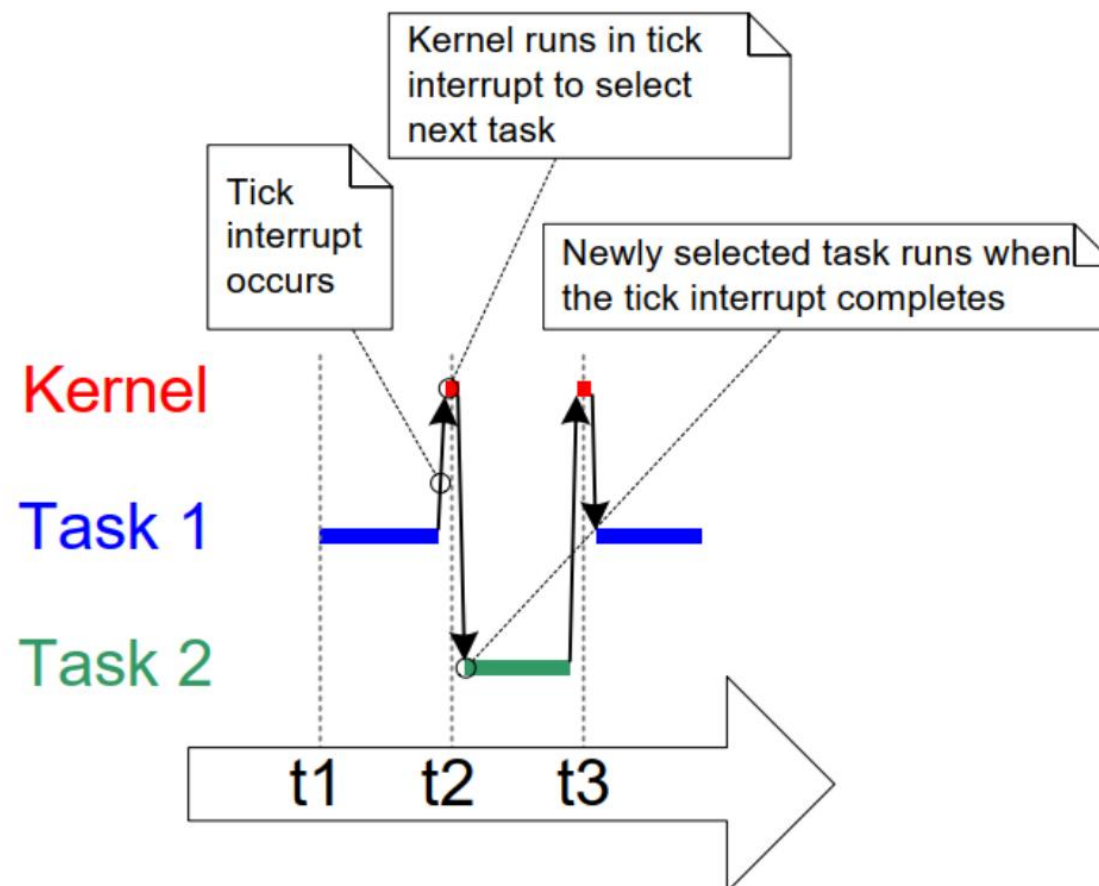


Figure 4.4 The execution sequence expanded to show the tick interrupt executing

2) FREERTOS

FreeRTOS API calls specify time in multiples of tick periods, often referred to simply as 'ticks'. The `pdMS_TO_TICKS()` macro converts a time specified in milliseconds into a time specified in ticks. The resolution available depends on the defined tick frequency, and `pdMS_TO_TICKS()` cannot be used if the tick frequency is above 1KHz (if `configTICK_RATE_HZ` is greater than 1000). Listing 4.10 shows how to use `pdMS_TO_TICKS()` to convert a time specified as 200 milliseconds into an equivalent time specified in ticks.

```
/*
 * pdMS_TO_TICKS() takes a time in milliseconds as its only parameter,
 * and evaluates to the equivalent time in tick periods. This example shows
 * xTimeInTicks being set to the number of tick periods that are equivalent
 * to 200 milliseconds.
 */
TickType_t xTimeInTicks = pdMS_TO_TICKS( 200 );
```

Listing 4.10 Using the `pdMS_TO_TICKS()` macro to convert 200 milliseconds into an equivalent time in tick periods

Using `pdMS_TO_TICKS()` to specify times in milliseconds, rather than directly as ticks, ensures times specified within the application do not change if the tick frequency is changed.

2) FREERTOS

- Task 2 has higher priority than Task 1

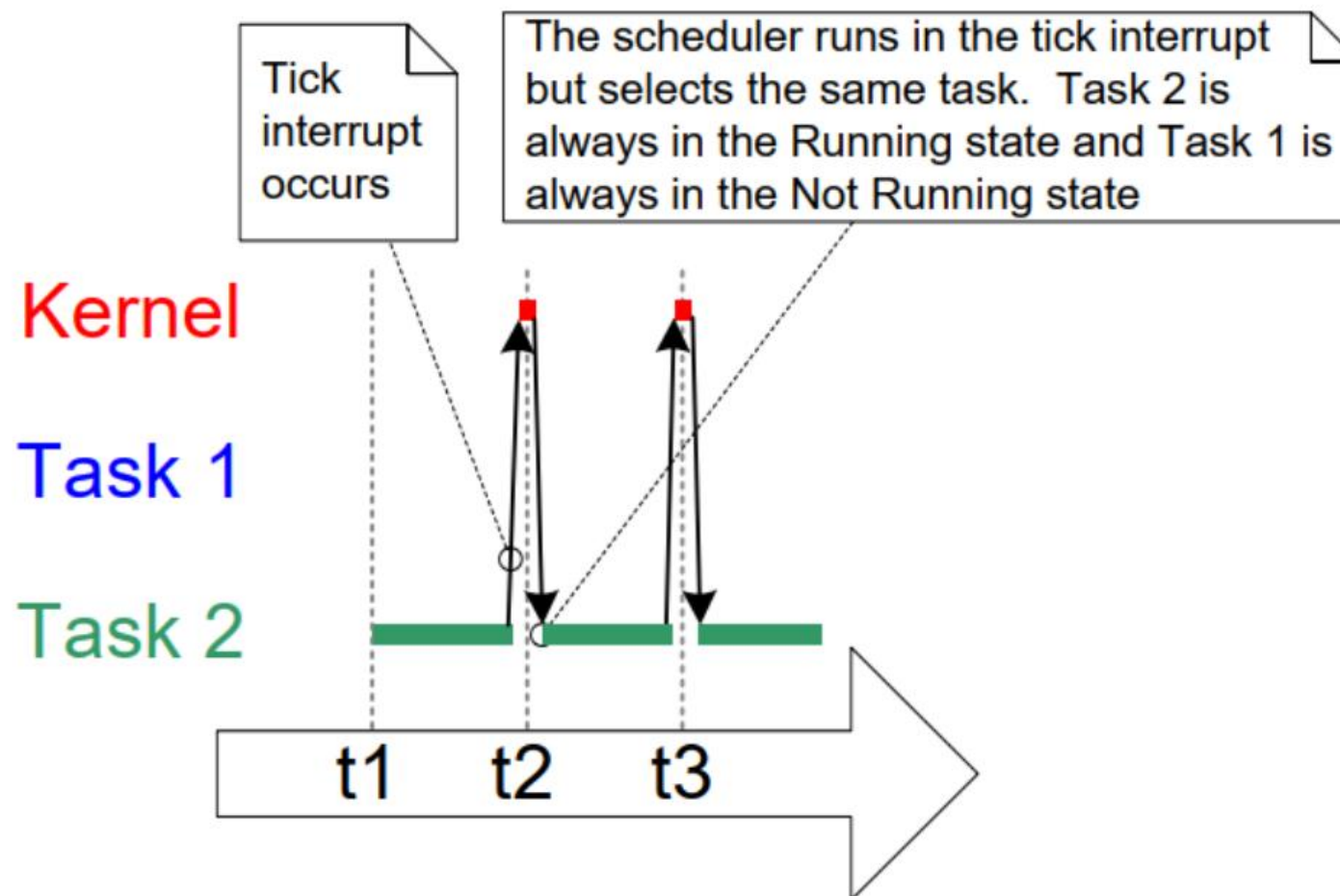


Figure 4.6 The execution pattern when one task has a higher priority than the other from Example 4.3

2) FREERTOS

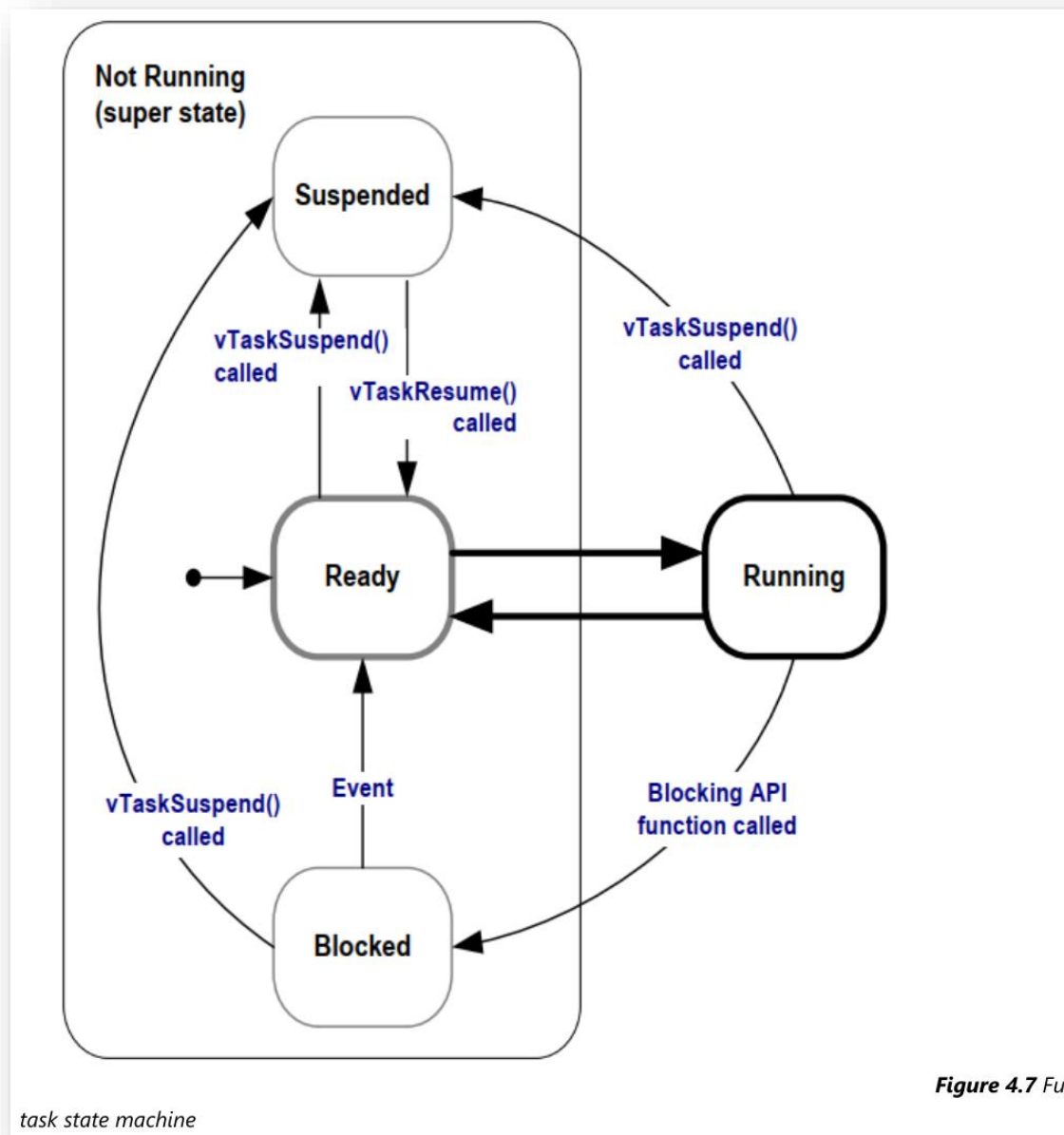


Figure 4.7 Full

2) FREERTOS

- In FreeRTOS, you can define **periodic** and **non-periodic** (continuous) tasks, the **scheduler** will **ensure** that the tasks will run in **real-time**

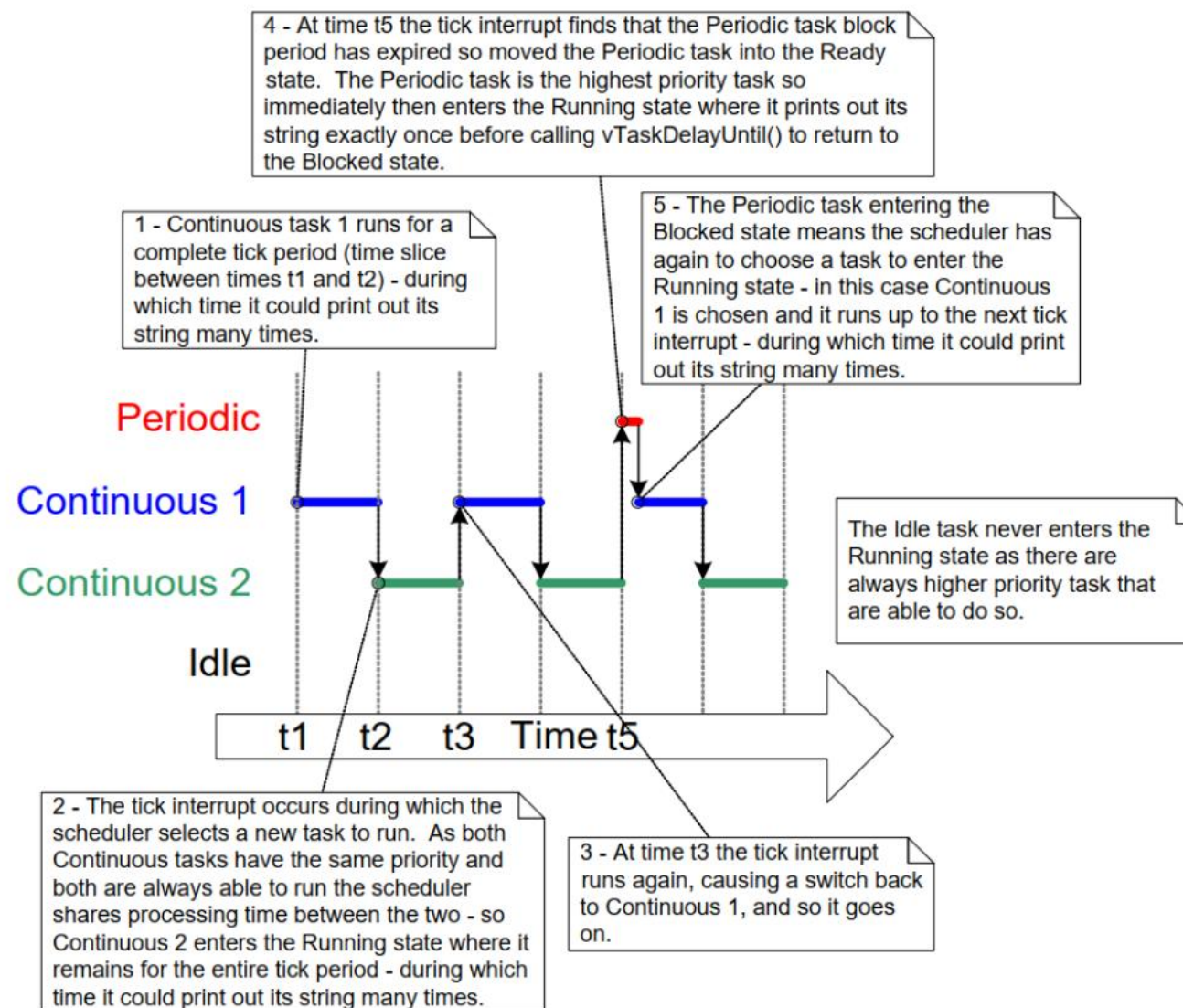
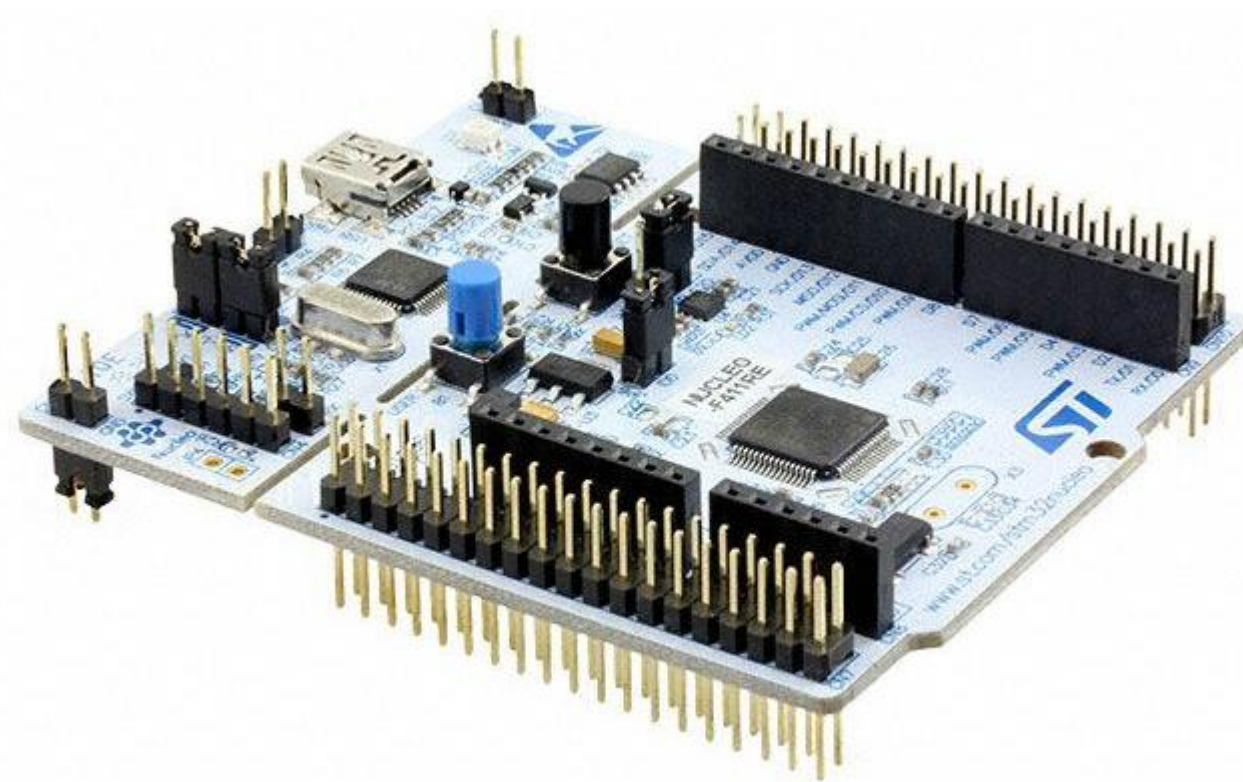


Figure 4.12 The execution pattern of Example 4.6

3) NUCLEO-64 STM32F401RE

- The Nucleo-64 STM32F401RE is a development board designed by STMicroelectronics

- Microcontroller:** It's built around the STM32F401RE microcontroller, which belongs to the STM32 family. This microcontroller features an ARM Cortex-M4 core and offers various peripherals, including GPIO pins, timers, ADCs, UARTs, SPI, I2C, etc.
- Form Factor:** The "Nucleo-64" refers to the board's form factor and pin count. It's designed in a 64-pin layout, allowing easy access to the microcontroller's I/O pins for connecting external components.
- Arduino Compatibility:** The Nucleo boards often offer Arduino Uno-like headers, enabling compatibility with a wide range of Arduino shields, making it convenient for users familiar with the Arduino ecosystem.
- Integrated Debugger/Programmer:** Many Nucleo boards come with an integrated ST-Link debugger/programmer, allowing users to program and debug their applications directly on the board without requiring an external programmer.
- Expansion and Compatibility:** These boards often provide extension connectors, enabling users to connect additional modules or shields for expanding the board's capabilities.



Source: chatGPT from openAI

You should now see a view of the IOC file configurator, like that in Figure 2. If not, double click the `.ioc` file in your projects' file list.



1. Middleware → FREERTOS
2. Interface → CMSIS_V1
3. Config parameters → ENABLE_FPU → Enabled
4. TICK_RATE_HZ → click line → read description, remember value
5. USE_TRACE_FACILITY → Enabled
6. Include parameters → vTaskDelayUntil → Enabled⁸
7. Advanced settings → USE_NEWLIB_REENTRANT → Enabled
8. Tasks and Queues → double click defaultTask → Priority → osPriorityLow
9. *Optional:* Look through the other different tabs in the FreeRTOS settings to see what's configurable.

10. Left side menu $\rightarrow$ System Core $\rightarrow$ SYS
11. Timebase Source $\rightarrow$ TIM1
12. If you are using the WB55⁹ then skip this step. Look at Figure 3 for guidance.
 1. Click on the Clock Configuration tab.
 2. Locate **PLL Source Mux**. Continuing to the right:
 3. /M $\rightarrow$ /8
 4. *N $\rightarrow$ X64
 5. /P $\rightarrow$ /4
 6. Verify that SYSCLK (MHz) is 32.
 7. APB1 Prescaler $\rightarrow$ /1
 8. Verify with Figure 3.
13. Press **Ctrl+S** to save.
14. You should¹⁰ be asked about code generation. Pick **Yes**.
15. Verify that you now have a function called `StartDefaultTask()` in `main.c`. It's also useful to look at the code surrounding the call to `osKernelStart()` in `main()`.

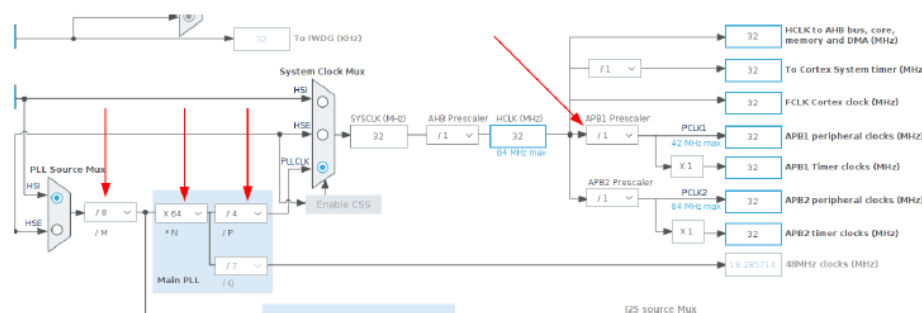


Figure 3: F401RE board **ONLY** - Red arrows point to where you should change values.

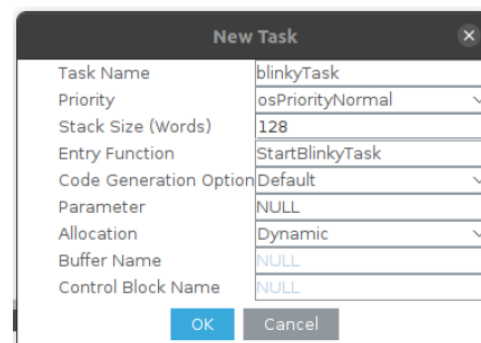
3) NUCLEO-64 STM32F401RE

5.3 Creating the Blinky Task

Now that everything is set up, let's make a LED blink.

1. IOC configuration tool → Pinout & Configuration tab
2. Middleware → FREERTOS → Tasks and Queues → Click Add under Tasks box (*not* Queues)
3. Only change these parameters, look in Figure 4 to verify:
 - Task Name: blinkyTask
 - Priority: osPriorityNormal
 - Entry Function: StartBlinkyTask
4. See Figure 5. Remember the identifier of one of the LEDs (e.g. LD2).
5. Save and generate code.
6. `main.c` → Outline (right side of CubeIDE window) → Click `StartBlinkyTask()` to go to function definition (there are two, one for declaration and one for definition)
7. **IMPORTANT:** Read Chapter 5.3.1.
8. Make `StartBlinkyTask()` look like that in Chapter 5.3.2.

Now proceed to Chapter 5.4 on how to compile and run your code.



New Task	
Task Name	blinkyTask
Priority	osPriorityNormal
Stack Size (Words)	128
Entry Function	StartBlinkyTask
Code Generation Option	Default
Parameter	NULL
Allocation	Dynamic
Buffer Name	NULL
Control Block Name	NULL
OK Cancel	

Figure 4: Adding a new task.

ANY QUESTIONS?